МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра "ПОВТ и АС"

Технология Java Server Faces

Практикум

по выполнению лабораторной работы № 5

по дисциплине "Объектно-ориентированное программирование"

Ростов-на-Дону

20 г.

Составитель: к.ф.-м.н., доц. Габрельян Б.В.

Практикум по выполнению лабораторной работы по дисциплине "Объектно-ориентированное программирование". ДГТУ, г. Ростов на-Дону, 20 г.

В практикуме кратко изложены теоретические вопросы, необходимые для успешного выполнения лабораторной работы, рабочее задание и контрольные вопросы для самопроверки.

Предназначено для обучающихся по направлениям 090304 "Программная инженерия", 020303 "Математическое обеспечение и администрирование информационных систем".

Ответственный за выпуск:

зав. кафедрой Долгов В.В.

Издательский центр ДГТУ, 20

Оглавление

Введение ... 4
Теоретическая часть ... 5
1. Базовые концепции ... 5
2. Управляемые компоненты и язык выражений ... 10
3. Компоненты пользовательского интерфейса ... 14
4. Составные компоненты ... 20
5. Базовые теги и теги фейслетов ... 23
6. Другие теги ... 28
7. Совместное использование JSF и JPA ... 28
Инструкция по технике безопасности при проведении работы ... 29
Цель работы ... 29
Рабочее задание ... 29
Общие положения ... 33
Материально-техническое обеспечение работы ... 33
Порядок выполнения работы ... 34
Контрольные вопросы ... 34
Перечень использованных информационных ресурсов ... 35

Введение

Java Server Faces (JSF) - это фреймворк для работы с пользовательским интерфейсом в web-приложениях Java. Эта технология разрабатывалась для борьбы со сложностью написания и сопровождения приложений, которые выполняются на сервере приложений Java и отображают свой пользовательский интерфейс на стороне клиента. Она позволяет упростить: создание пользовательского интерфейса за счет использования стандартных UI-компонентов, передачу данных от UI-компонентов и в UI-компоненты, управление состоянием пользовательского интерфейса между запросами, создание специфичных переиспользуемых UI-компонентов. Дает простую модель написания серверного кода обработки событий, генерируемых клиентом. Технология задает стандарт для вспомогательных средств, ориентированных на разные виды разработчиков. От "корпоративных", занимающихся созданием процедурного кода и бизнес-логики приложения, но не обязательно знающих объектно-ориентированное программирование, до "системных", знающих фундаментальные основы ООП и занимающихся разработкой повторно используемых программных компонентов. Корпоративные разработчики обычно зависят от средств разработки, а системным программистам может быть достаточно использовать текстовый редактор. Поэтому JSF разработана чтобы использоваться в средствах разработки, но предлагает также фреймворк и API, которые можно использовать без вспомогательных средств разработки. [1]

На сегодняшний день развитием технологии JSF занимается организация Eclipse Foundation. Начиная с версии JSF 3.0 имена пакетов в этой технологии (как и в других технологиях Jakarta EE) изменены с javax на jakarta.

Теоретическая часть

1. Базовые концепции

Основными концепциями JSF являются понятия фейслетов, управляемых компонентов (managed beans) и языка выражений (Expression Language - EL). Фейслеты представляют собой html-страницы со специальными тегами и встроенными выражениями EL. Существует набор стандартных тегов JSF, кроме того, можно использовать теги стандартной библиотеки JSTL, библиотеки тегов сторонних разработчиков и создавать свои теги, представляющие составные компоненты. Как и в JSTL стандартные теги JSF разнесены по разным пространствам имен, и чтобы их использовать на своих JSF-страницах (в своих фейслетах) нужно задать для этих пространств имен префиксы. Существуют договоренности о том, какие префиксы назначать стандартным пространствам имен JSF (и JSTL). В следующей таблице представлены стандартные пространства имен и назначаемые им по соглашению префиксы.

URI	Префикс	Описание
http://xmlns.jcp.org/jsf/core	f	Пользовательские действия (не связано с отрисовкой), например, f:selectItem, f:validateLength, f:convertNumber
http://xmlns.jcp.org/jsf/html	h	Компоненты и их html-представления, например, h:commandButton, h:inputText
http://xmlns.jcp.org/jsf/facelets	ui	Поддержка шаблонов, например, <ui:fragment>, <ui:define>, <ui:include>, <ui:insert>
http://xmlns.jcp.org/jsf/composite	composite	Объявление составных компонентов
http://xmlns.jcp.org/jsp/jstl/core	c	Базовая библиотека компонентов JSP, например, <c:forEach>, <c:if>

http://xmlns.jcp.org/jsp/jstl/functions	fn	Стандартная библиотека функций JSP, например, fn:length, fn:split, fn:contains, fn:substring

Задача фейслетов обеспечить те виды (представления), с которыми будет работать пользователь. Фейслеты содержат элементы графического пользовательского интерфейса (UI), но располагаются на сервере. При отсылке страницы пользователю фейслет преобразуется в обычную html-страницу. Преобразование выполняется в несколько фаз. При этом сохраняется состояние, которое пользователь задает или получает через элементы UI. Для хранения состояния используются объекты Java-классов (управляемые компоненты) аннотированные @Named или описанные в специальном файле-дескрипторе faces-config.xml. Связь между фейслетом (JSF-страницей) и управляемым компонентом осуществляется с помощью EL. Элементы UI обеспечивают навигацию по приложению, то есть возможность переходить с одной страницы на другую.

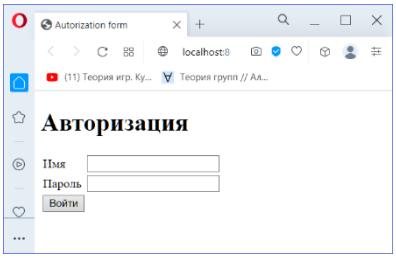
Пример. Приложение, выводящее форму авторизации и, в зависимости от того зарегистрирован пользователь или нет выполняющее переход на страницу авторизованного, либо неавторизованного пользователя. В архитектуре MVC модель будет отвечать за информацию о зарегистрированных пользователях, логику работы с пользователями и связь с уровнем данных. Представления – за отображение формы авторизации и страниц пользователей. Контроллер будет отвечать за связь вводимых пользователем данных с моделью. Технические вопросы организации вывода компонентов пользовательского интерфейса и получения информации от них обеспечивает технология JSF, для контроллера и модели создан класс UserModel (managed bean), для уровня данных класс User, представления реализованы как фейслеты index.xhtml (форма авторизации), reg_page.xhtml

(страница зарегистрированного пользователя) и unreg_page.xhtml (страница незарегистрированного пользователя).

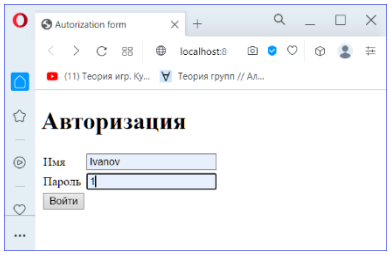
Для доступа к стартовой странице приложения в проекте IntelliJ Idea нужно зайти в Run/Edit Configurations… Server URL: и дописать index.jsf. Несмотря на то, что расширение для страницы xhtml, по умолчанию адреса, заканчивающиеся на jsf (*.jsf) автоматически отображаются на фейслеты (xhtml-страницы).

Файл webapp/WEB-INF/index.xhtml

```
 <!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
     xmlns:h="http://xmlns.jcp.org/jsf/html">
<head>
   <title>Autorization form</title>
</head>
<body>
<h1>Авторизация</h1>
<h:form>
   <h:panelGrid columns="2">
      <h:outputText value="Имя"/>
      <h:inputText value="#{usersModel.curUserName}"/>
      <h:outputText value="Пароль"/>
      <h:inputText value="#{usersModel.curUserPsw}"/>
      <h:commandButton value="Войти" action="#{usersModel.nextPage()}"/>
   </h:panelGrid>
 </h:form>
</body>
</html>
```

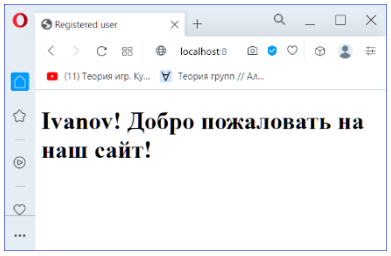


В полях ввода (inputText) отображаются имя и пароль текущего пользователя из модели (свойства UsersModel, изначально пустые строки). После ввода пользователем новых значений и отправки формы на сервер (commandButton "Войти") свойства curUserName и curUserPsw управляемого



компонента UsersModel получают эти значения, затем вызывается метод модели nextPage() (атрибут action). Этот метод проверяет, зарегистрирован ли пользователь в модели и, в зависимости от результата проверки возвращает строку с именем файла, то есть страницы, на которую произойдет переход. Возвращаемая строка должна содержать только имя файла так как расширение xhtml предполагается по умолчанию.

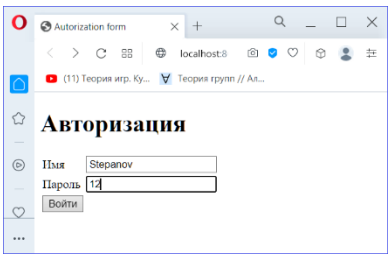
Файл webapp/pages/reg_page.xhtml



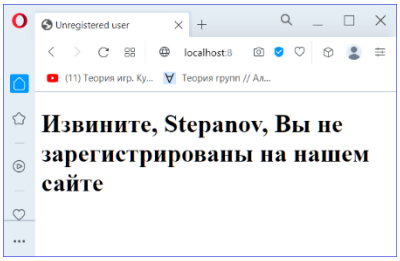

```
<!DOCTYPE html>
 <html xmlns="http://www.w3.org/1999/xhtml">
<head>
   <title>Registered user</title>
</head>
<body>
   <h1>#{usersModel.curUserName}! Добро пожаловать на наш сайт!</h1>
</body>
</html>
```

Для зарегистрированного пользователя выводится его имя и приветствие.

Файл webapp/pages/unreg_page.xhtml




```
 <!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
   <title>Unregistered user</title>
</head>
<body>
<h1>Извините, #{usersModel.curUserName}, Вы не зарегистрированы на нашем сайте</h1>
 </body>
</html>
```



Для незарегистрированного пользователя выводится его имя и сообщение о том, что он не зарегистрирован.

Файл User.java

```
package mypackage;
public class User {
```

```
    private String name;
    private String password;
    public User(String n, String p) {
        name = n;
        password = p;
    }
    public User() {
        this("", "");
    }
    public String getName() { return name; }
    public void setName(String n) { name = n; }
    public String getPassword() { return password; }
    public void setPassword(String p) { password = p; }
}
```

Представляет данные о зарегистрированном пользователе. Это то, что составляет уровень данных приложения. Если нужно хранить информацию о пользователях постоянно, информация из объектов класса User должна сохраняться в файле (или файлах), например, в реляционной базе данных, тогда можно использовать технологию JPA и User должен быть объявлен сущностью (@Entity).

Файл UsersModel.java

```
package mypackage;
import javax.enterprise.context.RequestScoped;
import javax.inject.Named;
import java.util.ArrayList;
import java.util.List;
@Named
@RequestScoped
public class UsersModel implements Serializable {
    private String curUserName, curUserPsw;
    List<User> registered = new ArrayList<>();
    {
        registered.add(new User("Ivanov", "1"));
        registered.add(new User("Petrov", "2"));
        registered.add(new User("Sidorov", "3"));
    }
    public String getCurUserName() {
```

```
        return curUserName;
    }
    public void setCurUserName(String cun) {
        curUserName = cun;
    }
    public String getCurUserPsw() {
        return curUserPsw;
    }
    public void setCurUserPsw(String cup) {
        curUserPsw = cup;
    }
    public boolean isRegistered(String n, String p) {
        for(int i=0; i<registered.size(); ++i) {
            User user = registered.get(i);
            if(user.getName().equalsIgnoreCase(n) && user.getPassword().equals(p))
                return true;
        }
        return false;
    }
    public String nextPage() {
        if(isRegistered(curUserName, curUserPsw))
            return "pages/reg_page";
        return "pages/unreg_page";
    }
}
```

Класс UsersModel как управляемый компонент обеспечивает связь своих свойств с данными, вводимыми пользователем, как модель обеспечивает связь с уровнем данных (список зарегистрированных пользователей) и бизнес-логику работы приложения.

2. Управляемые компоненты и язык выражений

Значения, относящиеся к разным компонентам пользовательского интерфейса можно связать со свойствами объектов классов, построенных по технологии JavaBeans, а обработчики событий от этих компонентов можно

"привязать" к методам таких классов. Сделать это можно с помощью языка выражений EL.

Компоненты JavaBeans, которые могут использоваться на страницах JSF, называются управляемыми компонентами (бинами) или классами-подложками. Изначально, для того чтобы класс представлял управляемые бины, нужно было задать для него аннотацию @ManagedBean (или описать в файле-дескрипторе faces-config.xml). Начиная с версии 2.2 аннотация @MangedBean (и соответствующие классы из пакета javax.faces.bean, а тем самым и сам этот пакет) объявлены устаревшими. Вместо этого применяется сквозная (то есть такая, которую можно использовать в разных Java EE технологиях) технология CDI, в которой определена аннотация @Named. Описанные таким образом классы представляют управляемые компоненты, так как создавать, уничтожать и обеспечивать изменение значений свойств объектов этих классов будет контейнер сервлетов. Можно использовать и обычные классы Java - POJO, но, разумеется, жизненным циклом этих объектов будет управлять программа, а не контейнер. Например,

```
import lombok.*;
…
// POJO
@Data                    // конструктор по умолчанию, get/set
public class User {
        private String name;
        private String password;
}
// управляемый компонент
@Named
public class UsersController {
        private String currentUser;
        private List<User> users = new ArrayList<>();
        … // конструктор по умолчанию
        public String getCurrentUser() { return currentUser; }
```

```
        …
        public List<User> getUsers() { return users; }
        …
        public String userPage(String uname) {
                return uname+"_page";
        }
}
```

Для UsersController также можно использовать аннотации проекта lombok. Объекты User создаются, уничтожаются программой, объекты UsersController контейнером.

Как долго существует управляемый компонент? По умолчанию пока активна текущая JSF-страница. Используя аннотации @ApplicationScoped, @SessionScoped, @RequestScoped можно задать другое время жизни компонента. Например,

```
@Named
@SessionScoped
public class UsersController implements Serializable {
        …
}
```

Компоненты-подложки (backed beans) - управляемые компоненты, предназначенные для связи с компонентами графического пользовательского интерфейса (хотя некоторые авторы не делают различия между управляемыми компонентами и компонентами-подложками) обычно должны существовать на протяжении обработки одного запроса.

Использовать управляемые компоненты на JSF-странице (в фейслете) можно с помощью языка выражений EL. Спецификация Expression Language (EL) предусматривает выражения-значения (value-expressions) и выражения-методы (method-expressions). Кроме того, вычисление выражения может быть непосредственным (immediate) или отложенным (deferred). В случае работы с управляемыми компонентами, выражение-значение обычно представлено

каким-то свойством этого компонента. Значение этого свойства или является частью выражения или непосредственно размещается на JSF-странице. Выражение-метод содержит вызов метода управляемого компонента. EL можно использовать в разных технологиях, в частности в JSP и в JSF. Выражения, используемые в JSP вида ${usersController.currentUser} вычисляются на раннем этапе обработки фейслета и представляют непосредственные (immediate) вычисления. В JSF, практически всегда используют отложенные (deferred) вычисления, выполняемые на поздних стадиях обработки, вида #{usersController.currentUser}. Это гарантирует, что свойство получит измененное значение.

В EL-выражениях можно использовать различные операции.

Арифметические: + - * / % (вместо / можно применять div, а вместо % mod).

Конкатенации строк: +=.

Логические: && (или and), || (или or), ! (или not).

Проверки условия: == (eq), != (ne), < (lt), > (gt), <= (le), >= (ge).

Проверки на пустоту (аргумент null или отсутствует): empty.

Условную: A ? B : C

(например, #{not empty usersController.curentUser ? 'Welcome' : 'Goodbye'}

Лямбда выражение: ->

например, #{((x, y) -> x + y)(3, 4)}

или #{f = (x, y) -> x + y; f(3, 4)}

Присваивания: =.

Точка с запятой: ; позволяет задать несколько выражений, например, #{2+3; 2*2}

Выражение-метод содержат вызов открытого (public) метода. При этом методу можно передавать параметры. Например,

#{usersController.userPage(usersController.currentUser)}

приводит к вызову метода userPage управляемого компонента UsersController и выводу на страницу строки, возвращенной этим методом.

3. Компоненты пользовательского интерфейса

Теги компонентов пользовательского интерфейса, или html-теги - это теги, которые на стадии прорисовки, то есть создания html-страницы, получаемой клиентом в качестве ответа на свой запрос отображаются на подходящие html-тэги. Эти теги определены в пространстве имен http://xmlns.jcp.org/jsf/html и используются со стандартным префиксом h. Для всех компонентов в теге можно задавать идентификатор (атрибут id), по которому в дальнейшем можно будет на него ссылаться. Некоторые теги практически аналогичны обычным тегам html, например, <h:html>, </h:html>, <h:body>, </h:body>, <h:head>, </h:head>. Другие представляют различные версии одного и того же тега html. Например,

<h:inputText> отображается на <input type="text"> - однострочное поле ввода,

<h:inputHidden> отображается на <input type="hidden"> - не отображается на странице, но, если размещается на форме, значение отсылается в запросе на сервер,

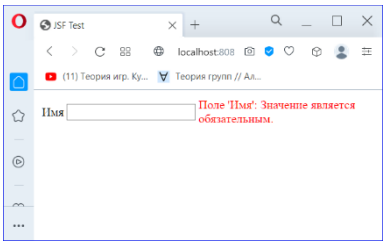
<h:inputSecret> отображается на <input type="password"> - при вводе отображаются звездочки.

Некоторые теги компонентов пользовательского интерфейса.

<h:inputText> - поле ввода, обычно связывается с некоторым свойством управляемого компонента. Основные атрибуты: value - обычно имя свойства компонента, required - может иметь значение "true" или "false", по умолчанию "false". Если установлен в "true" запрос с формы не будет отправлен до тех пор,

пока какой-то текст не будет введен в поле ввода и, если поле пустое и есть тег <h:message>, связанный с полем ввода через его идентификатор, может отображаться сообщение об ошибке. Атрибут label позволяет задать текст, который будет отображаться в сообщении об ошибке.

```
…
@Named
public class User {
        private String name;
        private String password;
        … // get/set
}
…
<!DOCTYPE html>
<html     xmlns:h="http://xmlns.jcp.org/jsf/html">
<body>
<h:panelGrid columns="3">
<h:outputText value="Имя"/>
<h:inputText      id="myinput"
                  value="#{user.name}"
                  required="true"
                  label="Поле 'Имя'">/>
<h:message for="myinput" style="color: red"/>
</h:panelGrid>
</body>
</html>
```



- отображается текущее значение свойства name (пустая строка), после ввода значение свойства содержит введенный текст. Текст ": Значение является обязательным" - это стандартное сообщение, определенное в JSF и выводимое на языке, заданном текущими языковыми настройками (локалью). Если не задать значение в атрибуте label - перед двоеточием будет выводиться мало читабельное имя компонента inputText, назначенное при обработке страницы.

<h:message> - отображает сообщение для компонента. У компонента должно быть задано значение атрибута id и это значение нужно указать в атрибуте for тега message.

<h:messages> - выводит сообщения для всех компонентов, а также глобальные сообщения, не связанные с каким-либо компонентом. Если атрибуту globalOnly тега messages задать значение true, то отображаться будут только глобальные сообщения.

<h:inputTextArea> - многострочное поле ввода.

<h:outputText> - выводит текст, заданный в атрибуте value.

<h:form> - создает форму, измененные значения компонентов которой будут обрабатываться на сервере при выполнении запроса. Атрибут method позволяет задать тип запроса (по умолчанию post). Атрибут action - запрашиваемую страницу.

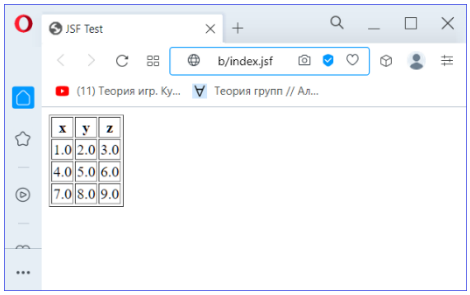
<h:panelGrid> - позволяет расположить компоненты в сетке состоящей из заданного числа столбцов (атрибут columns) и произвольного числа строк. См. пример выше. Компоненты заполняют столбцы в порядке их размещения в panelGrid. Чтобы разместить в одной ячейке сетки несколько компонентов нужно объявить их в теге <h:panelGroup>.

<h:dataTable> - строит таблицу с данными, полученными из некоторой коллекции. Коллекция задается в атрибуте value.

```
public class Point {
	private float x, y, z;
	… // get/set
}
@Named
public class Cluster {
	private List<Point> cluster = new ArrayList<>();
	public Cluster() {
		cluster.add(new Point(1,2,3));
		cluster.add(new Point(4,5,6));
		cluster.add(new Point(7,8,9));
	}
	public List<Point> getCluster() { return cluster; }
}
```

Фейслет

```
<!DOCTYPE html>
<html		xmlns:h=http://xmlns.jcp.org/jsf/html
		xmlns:f=http://xmlns.jcp.org/jsf/core>
<body>
<h:dataTable value="#{cluster.cluster}" var="p">
	<h:column>
		<f:facet name="header">x</f:facet>
		#{p.x}
	</h:column>
	<h:column>
		<f:facet name="header">y</f:facet>
		#{p.y}
	</h:column>
	<h:column>
		<f:facet name="header">z</f:facet>
		#{p.z}
	</h:column>
</h:dataTable>
</body>
</html>
```



Атрибут var позволяет задать имя переменной, которая поочередно будет связываться с элементами коллекции - объектами Point в данном примере.

<h:column> - задает данные для отдельного столбца таблицы. Здесь использован тег из другого пространства имен - пространства для базовых тегов JSF (http://xmlns.jcp.org/jsf/core). Тег <f:facet> позволяет для некоторых компонентов задать дополнительные элементы оформления. Для столбца таблицы это могут быть его заголовок (значение "header" атрибута name) или значение, выводимое после последнего элемента столбца - "подвал" (значение "footer" атрибута name).

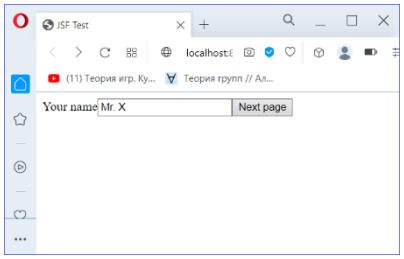
<h:commandButton>

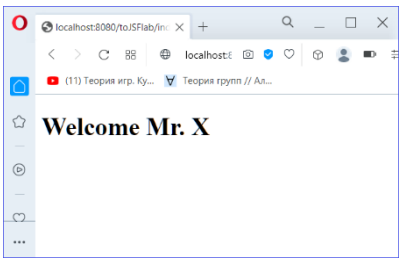
<h:button>

<h:commandLink>

<h:link> - компоненты, размещаемые на форме и инициирующие переход к другой странице. commandButton и button имеют вид кнопок, commandLink и link - вид гиперссылок. commandButton и commandLink выполняют запрос типа POST, button и link типа GET.

```
@Named
@SessionScoped
public class User implements Serializable {
   private String name;
…
}
 …
<h:form rendered="true">
   <h:outputText value="Your name"/>
   <h:inputText value="#{user.name}" />
   <h:commandButton
         value="Next page"
         action="resources/pages/hello"/>
</h:form>
…
```





<h:outputLink> - гиперссылка, не связанная с формой, позволяющая перейти на другую старницу.

<h:graphicImage> - отображает HTML-тег img.

<h:outputScript library="scripts" name="script.js"/> - загрузка файла JavaScript (script.js в данном примере) из указанного подкаталога (scripts в данном примере) стандартных ресурсов.

<h:outputStylesheet library="css" name="style.css"/> - загрузка CSS-файла (sttle.css в данном примере) из указанного подкаталога (css в данном примере) стандартных ресурсов.

<h:selectOneListBox> - отображает список выбора, в котором можно выбрать только один элемент.
<h:selectManyListBox> - отображает список выбора, в котором можно выбрать несколько элементов.

<h:selectOneMenu> - отображает выпадающее меню, в котором можно выбрать только один элемент.
<h:selectManyMenu> - отображает выпадающее меню, в котором можно выбрать несколько элементов.

<h:selectBooleanCheckbox> - отображает флажок с независимым выбором (checkbox).

<h:selectManyCheckbox> - отображает несколько флажков с независимым выбором (checkbox).

<h:selectOneRadio> - отображает несколько флажков с независимой фиксацией (radiobuttons).

4. Составные компоненты

Составной компонент - это компонент, состоящий из одного или нескольких JSF-компонентов, определенных в файле фейслета. Базируется на технологии Facelets и обработке ресурсов. Теги, позволяющие объявлять составные компоненты определены в пространстве имен http://xmlns.jcp.org/jsf/composite. Определение составного компонента состоит из двух частей: интерфейса и реализации. Интерфейс содержит описание атрибутов тега, представляющего составной компонент на JSF-странице. Атрибуты определяют вид и поведение составного компонента при его использовании. Составной компонент оформляется как фейслет - файл с расширением xhtml. Имя файла (без расширения) задает имя тега, представляющего этот составной элемент. Можно определить несколько составных тегов - библиотеку тегов. Она будет представлять собой каталог с xhtml-файлами. Этот каталог должен быть подкаталогом папки resources, расположенной в корневой папке приложения (в проекте IntelliJ Idea - webapp/resources). Например, составной компонент с тегом login [5] содержащий форму с метками Name и Password, двумя полями ввода и полями для вывода сообщений об ошибках проверки правильности введенных данных (наличия текста в полях ввода) и кнопкой для перехода на следующую JSF-страницу resources/pages/next_page.xhtml:

Файл **login.xhtml** из каталога **webapp/resources/mycomp**

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:cc="http://xmlns.jcp.org/jsf/composite">
<cc:interface>
   <cc:attribute name="name"/>
   <cc:attribute name="password"/>
   <cc:attribute name="actionListener"
        method-signature="void action(javax.faces.event.Event)"
        targets="ccform:loginButton"/>
</cc:interface>
<cc:implementation>
        <h:form id="ccform">
        <h:panelGrid columns="3">
                <h:outputText" value="Name">
                <h:inputText value="#{cc.attrs.name}" id="name"/>
                <h:message for="name" style="color: red"/>
                <h:outputText" value="Password">
                <h:inputText value="#{cc.attrs.password}" id="password"/>
                <h:message for="password" style="color: red"/>
        </h:panelGrid>
<h:commandButton id="loginButton" action="resources/pages/next_page"/>
        </h:form>
</cc:implementation>
</html>
```

Здесь для пространства имен составных компонентов задано имя cc. В интерфейсе объявлены атрибуты с именами name, password и actionListener. Последний представляет метод слушателя событий для компонента <h:commandButton>. Атрибут method-signature показывает, что это метод, не возвращающий значения и принимающий один аргумент типа ActionListener, объявленного в пакете javax.faces.event.Event (в случае Jakarta EE 9+ нужно заменить на jakarta.faces.event.Event). Этот слушатель событий должен быть связан с компонентом, имеющим идентификатор loginButton, причем сам этот компонент размещается на форме с идентификатором ccform - атрибут targets.

Доступ к значениям атрибутов, заданных в секции интерфейса в выражении EL можно получить через встроенный объект cc.attr (cc - от Composite Component). Свойствами этого объекта являются атрибуты нашего составного компонента, например, #{cc.attr.name}.

Управляемые компоненты (Managed Beans)

```
@Named
public class User {
	private String name;
	private String password;
	… // get/set
}

@Named
@SessionScope
public class UserService implements Serializable {
	private int count = 0;
	public void register(javax.faces.event.ActionEvent e) {
		++count;
	}
	public int getCount() { return count; }
}
```

Свойства User - name и password (совпадают с именами, заданными в атрибутах только ввиду логики их использования, синтаксически это могут быть любые допустимые идентификаторы) будут связываться со значениями соответствующих полей ввода компонента login. Метод register управляемого компонента UserService будет использоваться для прослушивания события ActionListener компонента <h:commandButton>. Сам компонент UserService должен быть доступен при переходе на следующую страницу, поэтому для него задается время жизни отличное от значения по умолчанию.

Использование тега login на JSF-странице

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:my="http://xmlns.jcp.org/jsf/composite/mycomp">
<body>
        …
        <my:login
                name="#{user.name}"
                password="#{user.password}"
                actionListener="#{userService.register}"/>
        …
</body>
</html>
```

Имя пространства имен для нашей библиотеки тегов составных компонентов строится из имени стандартного пространства имен http://xmlns.jcp.org/jsf/composite/ и имени каталога, содержащего объявления тегов. В данном случае mycomp.

5. Базовые теги и теги фейслетов

Базовые теги JSF определены в пространстве имен http://xmlns.jcp.org/jsf/core. Традиционно этому пространству имен назначается префикс f. Базовые теги представляют некоторые вспомогательные теги, которые используются в компонентах пользовательского интерфейса - в http-тегах, хотя технически они могут использоваться и вне этих тегов. Помимо тегов, рассмотренных в разделе "Компоненты пользовательского интерфейса" к базовым относятся также теги, позволяющие регистрировать блоки прослушивания для событий ActionListener, ValueChangeListener и т.п., использовать асинхронные запросы (AJAX), выполнять преобразования значений разных типов в строки и проверку допустимости задаваемых пользователем значений. Существуют стандартные преобразователи

(converters) и теги проверки правильности (validators), кроме того можно создавать свои конверторы и валидаторы.

<f:convertNumber> - преобразует значение компонента в котором указывается (в который вложен этот тег) в величину java.lang.Number, то есть в любой числовой тип. Предполагается, что полученное число будет помещено в соответствующее поле управляемого компонента. Как и в других конверторах с помощью атрибутов можно проводить различное форматирование полученного числа. Простейший вариант:

```
@Named
public class Person {
	private int age;
	private Data birthday;
	private String name;
	private String phone;
	… // get/set
}
…
<h:inputText value="#{person.age}">
	<f:convertNumber/>
</h:inputText>
```

- преобразует значение компонента в котором указывается (в который вложен этот тег) в величину java.lang.Date.

```
…
<h:inputText value="#{person.birthday}">
	<f:convertDateTime dateStyle="short">
</h:inputText>
…
```

<f:converter> - позволяет задать пользовательский преобразователь, преобразующий текст, содержащийся в компоненте в объект пользовательского класса. Технически преобразование должно выполняться классом, реализующим интерфейс javax.faces.convert.Converter (в примере ниже класс MyConverter). Имя этого класса должно быть указано в атрибуте converterId.

```
…
<h:inputText value="…">
        <f:converter converterId="MyConverter"/>
</h:inputText>
…
```

<f:validateRequired> - проверяет, что в поле ввода содержится какой-то текст, аналогичено использованию атрибута required компонента ввода.

```
…
<h:inputText value="#{person.age}">
        <f:valiadateRequired/>
</h:inputText>
…
```

<f:validateLength> - проверяет, что длина строки в поле ввода содержит нужное число символов, не менее чем значение, указанное в атрибуте minimum и не более чем значение атрибута maximum.

```
…
<h:inputText value="#{person.name}" required="true">
        <f:valiadateLength minimum="1" maximum="30"/>
</h:inputText>
…
```

<f:validateLongRange>

<f:validateDoubleRange> - проверяют, что значение поля ввода может быть преобразовано соответственно в java.lang.Long или java.lang.Double и это значение принадлежит диапазону, заданному атрибутами minimum и maximum.

```
…
<h:inputText value="#{person.age}">
```

```
        <f:valiadateLongRange minimum="14" maximum="65"/>
</h:inputText>
…
```

<f:validateRegex> - проверяют, что значение поля ввода соответствует регулярному выражению, заданному в атрибуте pattern.

```
…
<h:inputText value="#{person.phone}">
        <f:valiadateRegex pattern="8-\d{3}-\d{3}-\d{4}"/>
</h:inputText>
…
```

<f:validator> - позволяет задать пользовательский валидатор для проверки содержимого поля ввода. Технически преобразование должно выполняться классом, реализующим интерфейс javax.faces.validator.Validator и декорированным аннотацией @FacesValidator.

Теги фейслетов определены в пространстве имен http://xmlns.jcp.org/jsf/facelets. С помощью этого пространства имен можно создавать и использовать шаблоны страниц. В шаблоне логически выделяются отдельные части и им назначаются имена (тег <ui:insert>). Конкретные страницы - клиенты шаблона указывают, по какому шаблону они строятся (тег <ui:composition>) и представляют для именованных частей свои реализации (тег <ui:define>). Например, есть две страницы, содержимое которых разбивается на две части, каждая часть содержит какую-то свою информацию.

Файл layout.xhtml - шаблон

```
<!DOCTYPE html>
<html xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
```

```
<h:body>
        <h:panelGrid columns="1">
                <ui:insert name="part1">
                        This is part1
                </ui:insert>
                <ui:insert name="part2">
                        This is part2
                </ui:insert>
        </h:panelGrid>
</h:body>
</html>
```

Шаблон, содержащий сетку из одного столбца с двумя строками, первой строке назначено имя part1, а второй имя part2. Если клиент не задаст свою реализацию для какой-то части шаблона, то отобразится то, что указано в теле тега <ui:insert>, в данном случае строка "This is part1" или "This is part2".

Клиентская страница page1.xhtml

```
<!DOCTYPE html>
<html xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:ui="http://java.sun.com/jsf/facelets">
<ui:composition template="layout.xhtml">
   <ui:define name="part1">
      <h1>Welcome!</h1>
   </ui:define>
   <ui:define name="part2">
      <h:outputLink value="page2.xhtml">To page2</h:outputLink>
   </ui:define>
</ui:composition>
</html>
```

Построена по шаблону layout.xhtml, первая часть содержит строку "Welcome!", вторая ссылку на другую клиентскую страницу - page2.xhtml.

Клиентская страница page2.xhtml

```
<!DOCTYPE html>
```

```
<html xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:ui="http://java.sun.com/jsf/facelets">
<ui:composition template="layout.xhtml">
        <ui:define name="part2">
                <h1>Welcome!</h1>
        </ui:define>
</ui:composition>
</html>
```

Построена по тому же шаблону layout.xhtml, первая часть не реализована, поэтому будет отображаться строка "This is part1", объявленная в шаблоне, вторая часть содержит строку "Welcome!".

6. Другие теги

JSF позволяет использовать сквозные атрибуты (pass through attributes), позволяющие задавать в компонентах произвольные пары имя-значение, которые передаются без их интерпретации (см. [1]).

Можно использовать базовые теги и функции JSTL из пространств имен http://xmlns.jcp.org/jsp/jstl/core и http://xmlns.jcp.org/jsp/jstl/functions.

7. Совместное использование JSF и JPA

Как управляемые компоненты JSF, так и сущности JPA это объекты стандартных Java-классов. Поэтому сущность может одновременно быть и управляемым компонентом.

```
@Named
@RequestScoped
@Entity
@Table(name="users")
public class User implements Serializable {
        @Id
```

```
        private int id;
        …
}
```

Инструкция по технике безопасности при проведении работы

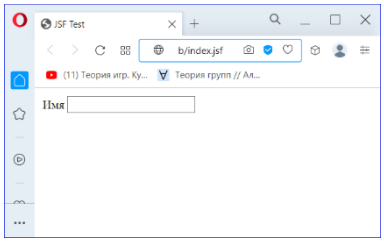
При выполнении работы нужно выполнять требования по технике безопасности при работе с электронными приборами.

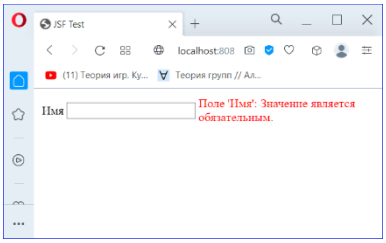
Цель работы

Изучение технологии Java Server Faces (JSF). Получение навыков проектирования и реализации web-приложений Java на основе технологии JSF.

Рабочее задание

ЗАДАНИЕ 1. Создайте собственный составной тег, представляющий компонент пользовательского интерфейса "именованный ввод" - текстовую строку (метку), поле ввода и поле для вывода сообщения (красным цветом) о неправильном вводе - отсутствии текста в поле ввода (<h:message>). Конкретное значение для метки должно задаваться через атрибут, например, label. Через другой атрибут, например, property, содержимое поля ввода должно связываться со свойством (тип String) какого-то компонента Java Beans. В итоге, если тег называется namedinput и его объявление помещено в каталог resources/mytaglib,





использоваться вновь созданный составной компонент должен подобно следующему:

```
@Named
public class MyBean {
        private String msg;
        … // get/set
}
… // в фейслете
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
     xmlns:h="http://xmlns.jcp.org/jsf/html"
     xmlns:my="http://xmlns.jcp.org/jsf/composite/mytaglib">
…
<body>
        <h:form>
                <my:namedinput label="Имя" property="#{myBean.msg}"/>
                …
                <h:commandButton …/>
        </h:form>
</body>
</html>
```

<u>ЗАДАНИЕ 2</u>. Создайте JSF-страницы (фейслеты), представляющие 1) форму авторизации и 2) регистрационную форму для пользователей электронного магазина. Форма авторизации должна содержать ссылку на регистрационную форму и наоборот. Создайте управляемый

компонент (managed bean) для хранения информации о зарегистрированном пользователе. Например,

```
public class User {
    private String name;
    …
}
```

```
@Named
public class RegUser extends User {
    …
}
```

При регистрации пользователь должен заполнить поля "Имя", "e-mail", "телефон", "пароль" и нажать кнопку "Подтвердить". Обеспечьте проверку правильности вводимых данных. Должны проверяться наличие имени и пароля, а также допустимость введенных значений e-mail и телефона. Используйте составной компонент, созданный при выполнении задания 1.

ЗАДАНИЕ 3. Используя теги из пространства имен http://xmlns.jsp.org/jsf/facelets создайте шаблон страницы, в котором предусмотрены три части: верхняя - заголовок, средняя - содержимое и нижняя - "подвал". Содержимое многих страниц в разных приложениях соответствует такому шаблону. Создайте клиентскую страницу, использующую данный шаблон и представляющую страницу электронного магазина некоторой фирмы. Заголовок шаблона должен содержать логотип фирмы (изображение в формате jpeg, png, gif), название этой фирмы и имя пользователя, зашедшего на сайт. Для тестирования, в средней части можно разместить текст, например, "Приветствуем Вас на стартовой странице нашего сайта!". Нижняя часть может содержать информацию о контактах для связи с администрацией фирмы - телефоны и e-mail адреса.

ЗАДАНИЕ 4. Спроектировать web-приложение для владельцев и пользователей небольших фирм по прокату автомобилей. Все пользователи проходят авторизацию (используйте реализацию из задания 2). Для незарегистрированных пользователей нужно предусмотреть регистрацию (используйте реализацию из задания 2). Есть два вида пользователей (две роли) - менеджеры самой фирмы и клиенты, абонирующие автомобили. После успешной авторизации пользователь попадает на свою страницу - страницу менеджера или страницу клиента. Менеджер может редактировать список доступных для проката автомобилей: добавлять, изменять, удалять информацию (название, цвет, серийный номер, информация о состоянии, например, новый или после ремонта, стоимость одного дня аренды), просматривать список арендованных на данный момент автомобилей с информацией о сроках аренды. Клиент видит список арендованных им на данный момент автомобилей с информацией о сроке аренды и ее общей цене, и список доступных автомобилей, может удалить автомобиль из первого списка и выбрать автомобиль из второго. В последнем случае он видит форму с информацией о выбранном автомобиле и должен задать срок (количество дней), на который хочет арендовать этот автомобиль. Клиент должен видеть общую сумму аренды и ему доступна кнопка "Арендовать". В данном случае предполагаем, что все клиенты джентльмены, поэтому нажатая кнопка означает, что оплата проведена. Используйте архитектуру MVC. Представления - фейслеты (все строятся по единому шаблону из

задания 3), контроллер - объекты-подложки, модель - управляемые компоненты и уровень данных.

ЗАДАНИЕ 5. Реализуйте приложение из задания 4. При разработке и для тестирования логики самого приложения можно использовать для хранения данных простые Java-объекты (POJO) и коллекции таких объектов. Но, затем, нужно обеспечить долговременное хранение информации в базе данных, используя технологию JPA.

Общие положения

Технология JSF построена в соответствии с архитектурой MVC и, вместе с другими технологиями Jakarta EE / Java EE (управляемы компоненты, язык выражений EL, внедрение контекстов и зависимостей CDI, проверка правильности компонентов Bean Validation, JPA) предлагает поддержку при создании всех уровней web-приложения. Важно правильно спроектировать архитектуру своего приложения в соответствии с подходом, предлагаемым JSF. Поэтому, помимо изучения собственно JSF, лабораторная работа призвана также закрепить навыки проектирования web-приложений с разделением в программе представления, обработки взаимодействия с пользователем, бизнес логики решаемой задачи и использования внешних источников данных.

Материально-техническое обеспечение работы

Компьютеры с установленным программным обеспечением:

1. Java SDK 11+.

2. IDE для работы с корпоративными Java-проектами - желательно IntelliJ Idea Enterprise.

3. Сервер приложений с поддержкой Java EE/Jakarta EE - желательно Apache TomEE+ 8+.

4. Сервер реляционных баз данных (СУБД), например, PostgreSQL.

5. ConnectorJ - драйвер для работы с сервером реляционных баз данных (например, PostgreSQL) из Java-программ.

Порядок выполнения работы

1. Ознакомиться с теоретическим материалом.

2. Выполнять задания в порядке их следования так как при выполнении текущего задания потребуются результаты, полученные при выполнении предыдущих заданий.

Контрольные вопросы

1. Что такое JSF?
2. Что такое управляемый компонент, компонент-подложка?
3. Какие три типа тегов для текстовых полей существуют в JSF?
4. На что указывает аннотация @ApplicationScoped?
5. Какие различные типы навигации по страницам поддерживаются в JSF?
6. Какие фазы жизненного цикла в JSF вы знаете?
7. Объясните назначение тега <h:form>.
8. Какие компоненты используются для отображения данных в табличном виде?
9. В чем разница между <h:dataTable> и <h:panelGrid>?

10. В чем разница между <h:commandButton> и <h:button>?

11. В чем разница между <h:commandLink> и <h:link>?

12. Какие различные типы событий JSF вы знаете?

13. Назовите несколько фейслет-тегов.

14. Какие различные типы проверки правильности (валидации) используются в JSF?

15. Какие различные типы выражений поддерживаются в JSF EL (Expression Language)?

16. В чем разница между мгновенными (immediate) и отложенными (deffered) выражениями?

17. Приведите пример использования <h:outputStylesheet>.

18. Расскажите о @ViewScoped, @SessionScoped и @RequestScoped аннотациях.

19. Какие существуют способы объявить класс управляемым компонентом JSF?

20. Какие теги для проверки правильности (валидации) существуют в JSF?

21. Какие различные теги JSF используются для преобразования типов?

22. Какие стандартные библиотеки тегов JSF вы знаете?

23. Как отобразить сообщения об ошибках в JSF?

24. Объясните в чем назначение атрибута rendered?

25. Может ли отсутствовать faces-config.xml у JSF приложения?

Перечень использованных информационных ресурсов

1. Jakarta EE. Jakarta Server Faces Specification. Version 3.0. Jakarta Server Faces Team, https://projects.eclipse.org/projects/ee4j.faces. – 2020, 453 p.

2. Jakarta EE. Jakarta Expression Language Specification. Version 4.0. Jakarta Expression Language Team, https://projects.eclipse.org/projects/ee4j.el. – 2020, 63 p.
3. Э.Гонсалвес "Изучаем Java EE 7". – СПб: Питер. – 2016, 640с.
4. Д.Хеффельфингер "Java EE 6 и сервер приложений GlassFish 3". – М.:ДМК-Пресс. – 2013, 416 с.
5. А.Гупта "Java EE 7. Основы". – М.:Вильямс. – 2014, 336 с.
6. L.Manelli, G.Zambon "Beginning Jakarta EE Web Development: Using JSP, JSF, MySQL, and Apache Tomcat for Building Java Web Applications". – Apress. – 2020, 416 с.
7. P.Spath "Beginning Jakarta EE: Enterprise Edition for Java: From Novice to Professional", 2-d ed. – Apress. – 2019, 454с.

Редактор А.А. Литвинова

ЛР № 04779 от 18.05.01.	В набор	В печать
Объем 0,5 усл.п.л., уч.-изд.л.	Офсет.	Формат 60x84/16.
Бумага тип №3.	Заказ №	Тираж 120. Цена

Издательский центр ДГТУ

Адрес университета и полиграфического предприятия:

344010, г. Ростов-на-Дону, пл. Гагарина, 1.